COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	10/06/26	**DATE SUBMITTED:**	10/07/26
MEMBER/S:	Egar, Jarel Dave Medina, Jomz Ryle E.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 11.1 Graphs

1. PROCEDURE OUTPUT

ILO A: Measure the runtime of algorithms using theoretical analysis

Assess if two arrays have any common values.

```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i])!= - 1){
            return false;
        }
    }
    return true;
}
```

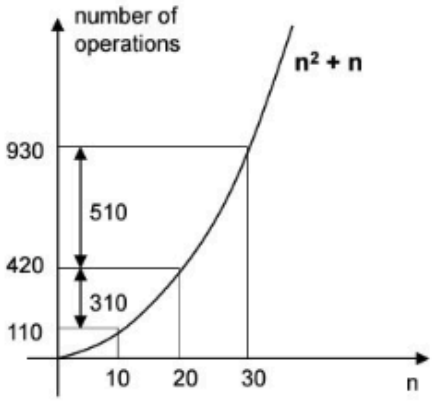
Performing worst case analysis:

- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparisons.

Given the above information, the total number of comparisons in the worst case is: _____

- Assume that m = n. (The arrays are the same size)

Provide an analysis of the graph for the worse case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Imagine that it is flu season and health department officials are planning to visit a school to ensure that all the enrolled children are administered their flu shot. However, there is a problem: a few children have already taken their flu shots but do not remember if they have been vaccinated against the specific category of flu that the health officials plan to vaccinate all the students against. Official records are sought out and the department is able to find a list of students that have already been administered the vaccine. A small excerpt of the list is shown here:

First Name	Last Name	Flu Shot?
1	3	Yes
2	2	No
2	3	Yes

Assume that all the names are positive integers and that the given list is sorted. Your task is to write a program that can look up the vaccination status of a given student in the list and outputs to the officials whether the student needs to be vaccinated. Students need to be vaccinated in case of two conditions:

- If they are not present in the list
- If they are present in the list but have not been administered a flu shot

Since the list can have a large number of students, your program should be as fast and efficient as possible. The final output of your program should look as follows:

```
Time taken to search = 45 microseconds
Student (836 118) needs vaccination.
Time taken to search = 37 microseconds
Student (836 118) needs vaccination.
Time taken to search = 53 microseconds
Student (836 118) needs vaccination.
```

The high-level steps for this implementation are as follows:

1. Represent each student as an object of the `Student` class.
2. Overload the required operators for the Student class so that a vector of students can be sorted using the STL `std::sort()` function.
3. Use a **binary search** to see if the student is present on the list.
4. If the student isn't present in the list, your function should return `true` since the student needs to be administered the vaccine.
5. Otherwise, if the student is present in the list but has not been administered the vaccine, return true.
6. Else, return false.

Step 0: Begin by including the following headers:

```
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <numeric>
```

Step 1: Define the STUDENT class as follows:

```
class Student{
    private:
        std::pair<int, int> name;
        bool vaccinated;

    public:
        //constructor
        Student(std::pair<int, int> n, bool v) : name(n), vaccinated(v){
        }

        // Getters
        auto get_name() { return name; }
        auto is_vaccinated() { return vaccinated; }
```

Step 2: Overload the required operators for the Student class so that a vector of students can be sorted using the STL `std::sort()` function.

```
        // Two people are same if they have the same name
        bool operator ==(const Student& p) const {
            return this->name == p.name;
        }

        // The ordering of a set of people is defined by their name
        bool operator< (const Student& p) const {
            return this->name < p.name;
        }

        bool operator> (const Student& p) const {
            return this->name > p.name;
        }
};
```

Step 3: The following function lets us generate a student from random data:

```
auto generate_random_Student(int max) {
    std::random_device rd;
    std::mt19937 rand(rd());

    // the IDs of Student should be in range [1, max]
    std::uniform_int_distribution<std::mt19937::result_type> uniform_dist(1, max);

    // Generate random credentials
    auto random_name = std::make_pair(uniform_dist(rand), uniform_dist(rand));
    bool is_vaccinated = uniform_dist(rand) % 2 ? true : false;
    return Student(random_name, is_vaccinated);
}
```

For the next important function, we will implement the use of **std::chrono for experimental analysis.**

The following code is used to run and test the output of our implementation.

```
void search_test(int size, Student p) {
    std::vector<Student> people;

    // Create a list of random people
    for (auto i = 0; i < size; i++)
        people.push_back(generate_random_Student(size));

    std::sort(people.begin(), people.end());

    // To measure the time taken, start the clock
    std::chrono::steady_clock::time_point                    begin                    =
std::chrono::steady_clock::now();
    bool search_result = needs_vaccination(p, people);

    // Stop the clock
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();
    std::cout << "Time taken to search = " <<
        std::chrono::duration_cast<std::chrono::microseconds>
        (end - begin).count() << " microseconds" << std::endl;
    if (search_result)
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
<< ") " << "needs vaccination." << std::endl;
    else
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
<< ") " << "does not need vaccination." << std::endl;
}
```

5. The following function implements our logic for whether a vaccination is needed:

```
bool needs_vaccination(Student P, std::vector<Student>& people) {
    auto first = people.begin();
    auto last = people.end();
    while (true)
    {
        auto range_length = std::distance(first, last);
        auto mid_element_index = std::floor(range_length / 2);
        auto mid_element = *(first + mid_element_index);

        // Return true if the Student is found in the sequence and
        // he/she's not vaccinated
        if (mid_element == P && mid_element.is_vaccinated() == false)
            return true;
            else if (mid_element == P && mid_element.is_vaccinated() == true)
            return false;
        else if (mid_element > P)
            std::advance(last, -mid_element_index);
        if (mid_element < P)
            std::advance(first, mid_element_index);

        // Student not found in the sequence and therefore should be vaccinated
        if (range_length == 1)
            return true;
    }
}
```

6. Finally the driver code is implemented as follows:

```
int main() {
    // Generate a Student to search
    auto p = generate_random_Student(1000);
    search_test(1000, p);
    search_test(10000, p);
    search_test(100000, p);
    return 0;
}
```

Fill out the table below and include in section 6 table 11-1.

Input Size (N)	Execution Speed	Screenshot	Observation(s)

Provide your own analysis of the outcome listed in this table.

ILO A Output/s:
If at the worst case, where both of the arrays have no common values and each elements of **n** in array **y** as well as **m** in array **x** are compared with each other, then the total number of comparisons would logically be '**mn**'. And when both arrays have equal lengths, then **m** would be equal to **n** and **mn** would then be **n*n** and consequently, **n^2**. Therefore: Given the above information, the total number of comparisons in the worst case is, **therefore, n^2.**
Observations/Analysis of the Graph:
Egar:
The graph shows a worst case scenario of $n^2 + n$ based on the code it provided. The graph provides a visualization of how the time complexity worsens as the value of n increases. In the graph I noticed that as the size of the array increases the number of operations also increases since based on the worst case it provided, if we were to find the number of operations of an array size of 50 it will be $50^2 + 50 = 2{,}550$.
Medina:
The graph in ILO A shows that of the worst case space complexity for the algorithm where it shows how the number of operations grow as **n** increases. To me, it looks like it has an exponential-like curve but represents a quadratic equation where, as **n** grows, the program eventually gets more and more inefficient with the amount of operations rapidly booming and may lead to crashes to weaker computers just as discussed in the lecture.

ILO B Code Output:

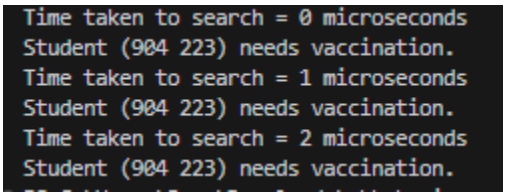
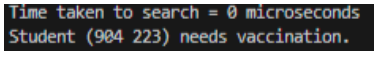
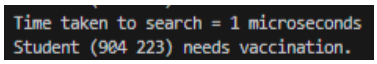
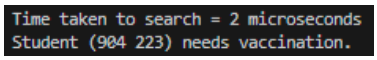


Table 11-1:

Input Size (N):	**Execution Speed:**	**Screenshot:**	**Observation(s):**
1000	0 microseconds		The binary search was completed instantly.
10000	1 microseconds		Completed slower than n at 1000 as the input size increases to 10000.
100000	2 microseconds		Only takes 0.2 microseconds at size 100000, proving that the code used is efficient with larger values of **n**.

Observations/Analysis:

Egar:

This ILO provided us with a direct implementation for Basic Algorithm Analysis. The code uses a binary search to demonstrate how long it will take each input size (1000,10000,100000) to get its desired output. I noticed during this ILO that the smaller the value of the size(N), the faster the system will finish its searching.

Medina:

ILO B shows us the implementation of a time function to measure how long the program takes to execute its code while comparing it to differing, large sizes of **n**. Additionally, the code implemented an efficient searching algorithm which looks like something that resembles a binary search algorithm to me. This is only confirmed further with how fast its execution speed is within the output, reaching only microseconds to complete.

2. SUPPLEMENTARY OUTPUT

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

For each of the problems below, provide the following:

- Theoretical Analysis
- Experimental Analysis
- Analysis and Comparison

Problem 1: Consider a program that returns true if the elements in an array are unique.

```
Algorithm Unique(A)
  for i = 0 to n-1 do
    for j = i+1 to n-1 do
      if A[i] equals to A[j] then
        return false
  return true
```

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.

```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3   y = brpower(x, (n-1)/2)
4   return x*y*y
5 if n is even then
6   y = brpower(x, n/2)
7   return y*y
```

SUPPLEMENTARY OUTPUTS:

Problem 1: Consider a program that returns true if the elements in an array are unique.

```
Algorithm Unique(A)
  for i = 0 to n-1 do
    for j = i+1 to n-1 do
      if A[i] equals to A[j] then
        return false
  return true
```

Theoretical Analysis:

This program shows the execution speed by using a nested loop. The worst-case scenario of this whole program is n^2 - n/2. Because it performs all comparisons in place using only loop counters, its space complexity remains constant at O(1).

Experimental Analysis:

Code:

EgarMedinaProb1.cpp

EGAR_HOA11.1 Prob1 > EgarMedinaProb1.cpp > main()

```cpp
#include <iostream>
#include <vector>
#include <chrono>


bool Unique(const std::vector<int>& A) {
    int n = A.size();

    // check every pair of elements
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (A[i] == A[j]) {
                return false; // Found a duplicate
            }
        }
    }

    return true;
}

void test_unique(int n) {
    // responsible for generating a vector of size n with unique elements
    std::vector<int> A;
    for (int i = 0; i < n; i++) {
        A.push_back(i);
    }

    // Measures the start time
    auto start = std::chrono::high_resolution_clock::now();

    bool result = Unique(A);

    // Measures the end time
    auto end = std::chrono::high_resolution_clock::now();

    // Calculate the duration in microseconds
    auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end - start).count();

    std::cout << "Input Size (N) = " << n << std::endl;
    std::cout << "Execution Speed = " << duration << " microseconds" << std::endl;
    std::cout << "Result = " << (result ? "true" : "false") << std::endl;
    std::cout << std::endl;
}

int main() {
    std::cout << "Problem 1: " << std::endl << std::endl;

    test_unique(1000);
    test_unique(10000);
    test_unique(100000);

    return 0;
}
```

Output:

```
Problem 1:

Input Size (N) = 1000
Execution Speed = 3279 microseconds
Result = true

Input Size (N) = 10000
Execution Speed = 327196 microseconds
Result = true

Input Size (N) = 100000
Execution Speed = 33080039 microseconds
Result = true
```

Analysis and Comparison:

This program measures the execution speed of element uniqueness checking by using a nested loop structure. The total number of comparisons required in the worst-case scenario is n^2 - n/2, resulting in quadratic time complexity O(n^2). Because the algorithm processes all comparisons directly in place using only basic loop indices, its space complexity remains constant at O(1).

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.

```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3   y = brpower(x, (n-1)/2)
4   return x*y*y
5 if n is even then
6   y = brpower(x, n/2)
7   return y*y
```

Theoretical Analysis:

For **rpower**, each recursive call makes it so that the exponent decreases by one until **n** is equal to 0 where **n** comes from the multiplying and **n + 1** per function call. Additionally, the recursive calls remain on the stack until the base case is reached, and therefore, both space and time complexity is **O(n)**.

In the case for **brpower**, recursive calls are repeated as well until the base case is reached where **n** is equal to 0, much like with **rpower**. However, this time, the exponent is halved with each call, resulting in both space and time complexities to be **O(logn)**.

Algorithms (both best and worst case):

rpower:

- Time Complexity: O(n)
- Space Complexity: O(n)

brpower:

- Time Complexity: O(logn)
- Space Complexity: O(logn)

Experimental Analysis:

Code:

main.cpp > main()

```cpp
#include <iostream>
#include <chrono>
#include <iomanip>

// 1st Algo
int rpower(int x, int n) {
    if (n == 0)
        return 1;

    return x * rpower(x, n - 1);
}

// 2nd Algo
int brpower(int x, int n) {
    if (n == 0)
        return 1;

    if (n % 2 != 0) {
        int y = brpower(x, (n - 1) / 2);
        return x * y * y;
    }

    int y = brpower(x, n / 2);
    return y * y;
}
```

```cpp
// way to test (structure taken from the lab activity)
void power_test(int x, int n) {
    const int repetitions = 2000;

    volatile int result = 0;

    std::cout << "\nBase (x): " << x
              << "\nExponent (n): " << n << '\n';

    std::chrono::steady_clock::time_point begin =
        std::chrono::steady_clock::now();

    for (int i = 0; i < repetitions; i++) {
        result = rpower(x, n);
    }

    std::chrono::steady_clock::time_point end =
        std::chrono::steady_clock::now();

    double rpower_time =
        std::chrono::duration<double, std::micro>(
            end - begin
        ).count() / repetitions;

    std::cout << "rpower result = " << result << '\n';
    std::cout << "Average time taken by rpower = " << rpower_time << " microseconds\n";


    begin = std::chrono::steady_clock::now();

    for (int i = 0; i < repetitions; i++) {
        result = brpower(x, n);
    }

    end = std::chrono::steady_clock::now();

    double brpower_time =
        std::chrono::duration<double, std::micro>(
            end - begin
        ).count() / repetitions;

    std::cout << "brpower result = " << result << '\n';
    std::cout << "Average time taken by brpower = " << brpower_time << " microseconds\n";
}
```

```cpp
int main() {
    std::cout << std::fixed << std::setprecision(4);

    int x = 1;

    power_test(x, 100);
    power_test(x, 1000);
    power_test(x, 10000);

    return 0;
}
```

Output:

```
Base (x): 1
Exponent (n): 100
rpower result = 1
Average time taken by rpower = 0.2404 microseconds
brpower result = 1
Average time taken by brpower = 0.0186 microseconds

Base (x): 1
Exponent (n): 1000
rpower result = 1
Average time taken by rpower = 2.2216 microseconds
brpower result = 1
Average time taken by brpower = 0.0383 microseconds

Base (x): 1
Exponent (n): 10000
rpower result = 1
Average time taken by rpower = 27.0483 microseconds
brpower result = 1
Average time taken by brpower = 0.0457 microseconds
```

Analysis and Comparison:

First, I'd like to address how the base that I used was 1 for the experimental analysis; this is to make sure that I don't go over the integer limit with how high it can get and my computer doesn't crash because I would rather not risk it. Continuing on, **rpower** seems to have increased a lot in execution time as the exponent grew from 100 to 10000. It matches the theoretical analysis of having a time complexity of **O(n)** as each added unit of the exponent needs another recursive call and multiplication. Furthermore, **brpower** increased only slightly compared to **rpower's** sudden rise. This also matches our theoretical time complexity, **O(logn)** as for each halving of the exponent, it reduces much of the recursive calls used. Overall, I believe that both the theoretical and experimental analysis match with both of the algorithms with **brpower** scaling better as the exponent increases.

Conclusion/s:
Egar:
This activity helped me understand how time and space complexity work and how they affect our computers in real time. In the procedures, ILO A and ILO B, we were tasked with analyzing the theoretical and experimental execution of the programs. In ILO A, we observed how operations scale quadratically, where an input size of 50 results in 2,550 operations based on the worst-case scenario of $n^2 + n$. In ILO B, we saw how binary search optimizes performance across larger inputs up to 100,000 by drastically reducing search steps. In Supplementary Problem 1, implementing the nested loops for element uniqueness checking demonstrated that a worst-case comparison count of $n^2 - n/2$ caused execution time to increase from 3,279 microseconds to 33 seconds while maintaining space complexity O(1). Meanwhile, in Problem 2, testing the rpower and the bpower confirmed that, with a base of 1, it safely prevented a system crash while also demonstrating that bpower's log growth is much more efficient than rpower's linear growth as the exponent increases. Overall, this activity showed me that matching theoretical predictions with real-world testing is essential for building fast software, as choosing the right algorithm can mean the difference between a program running instantly or taking seconds to execute.
Medina:
The activity has helped me in understanding the importance of time and space complexity, performing both theoretical and experimental analysis on your program, as well as comparing them for any discrepancies that might reveal bugs that may be occurring despite running your program successfully. Additionally, I also now know the significance of counting each of the operations as it gives insights on the difference of execution time with the results showing how much brpower became much more efficient in the supplementary activity as the exponent is reduced in halves rather than by one with rpower. Furthermore, I'd like to add that although the theoretical complexity may represent the growth of an algorithm, experimental testing may have differing results depending on the computer used and its testing conditions.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **None**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the

permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.